

ТЕМА: РЕКУРСИЯ И СТЕК



ВВЕДЕНИЕ

В программировании невозможно обойтись без 3 вещей: ошибок, рекурсии и стека. Здесь мы не будем допускать ошибок, но изучим что такое рекурсия и как с ним связан стек

УРОК: БИНАРНЫЙ ПОИСК - РЕАЛИЗАЦИЯ ПО РЕКУРСИИ

Бинарный поиск

это эффективный алгоритм поиска элемента в отсортированном массиве, использующий рекурсию. Он делит массив на две части и рекурсивно ищет в нужной части



Принцип работы

- 01 Определите начальный (low) и конечный (high) индексы массива
- 02 Найдите средний индекс (mid) и сравните элемент в этом индексе с искомым значением
- 03 Если элемент равен искомому, верните его индекс
- 04 Если искомый элемент меньше, ищите в левой части массива
- 05 Если искомый элемент больше, ищите в правой части массива
- 06 Повторяйте до тех пор, пока low не станет больше high.

Пример кода с базовым случаем

```
def binary_search_recursive(arr, low, high, target):  
    if low <= high:  
        mid = (low + high) // 2  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            return binary_search_recursive(arr, mid + 1, high, target)  
        else:  
            return binary_search_recursive(arr, low, mid - 1, target)  
    else:  
        return -1 # Если элемент не найден  
  
# Пример использования:  
arr = [10, 20, 30, 40, 50]  
target = 30  
print("Индекс элемента:", binary_search_recursive(arr, 0, len(arr) - 1, target))
```

Характеристики

Сложность

Временная сложность: $O(\log n)$, где n — количество элементов в массиве

Пространственная сложность: $O(\log n)$ из-за рекурсии (длина стека вызовов)

Преимущества

Эффективность поиска в отсортированных структурах данных

Недостатки

Зависимость от сортировки массива и возможное переполнение стека при глубокой рекурсии

ВЫВОД

Мы рассмотрели основные концепции рекурсии. Эта тема является основой для понимания более сложных алгоритмов и структур данных, которые используются в программировании